AIM 2627 Linux Coursework

2026 年 10 月 6 日

0. 开始前

> **序章 · 上真车**
> IAMET-406 外，2027.3
> 在前面的 Python 作业里，你已经让一台理想化的哨兵学会了自检、看地图和巡逻。它在你自己的电脑上跑得很好，遇到障碍会绕开，发现敌人会做出决策，连输出报告都很整齐。
> 于是某位何动力驴说："接下来简单，那我们上真车试一下吧。"（骗你的如果这个时候才上真车那我们就没救了））
> 这句话说完的时候是晚上八点。按照我们对"接下来简单"的历史理解，大家已经开始查实验室附近哪家店能送凌晨的外卖了。
> 哨兵安静地停在角落。它没有屏幕，没有键盘，也没有义务知道你说的"那个文件夹"到底在哪。你打开终端，发现交接说明只有一份看起来就是 AI 写的 README
> 很好。代码还没来得及证明自己有问题，环境已经抢先一步了。
> 今晚，你接手的就是这台机器。先把交接做明白，再讨论它究竟能不能动。

聪明的你应该感受过我们 Python Coursework 的震撼美味，并且应该已经熟知了如何正确的提交一份 Assessment。

欢迎来到 Linux 的 Assessment。本作业不要求机器人硬件、ROS 或 Python 编程。你只填写 Bash 脚本，教师已提供训练程序、临时 SSH 服务和全部公开测试。**GitHub Actions 是唯一正式评估通道**。不交截图，不交人工操作记录，没有人工分、答辩分或独立 grader。

题号	题目	对应内容	分值
Q1	找到开发目录	路径、隐藏文件、文本读取	10
Q2	建立哨兵工作区	目录、空文件、多行文本	10
Q3	整理配置与权限	复制、移动、删除、权限、执行	10
Q4	从日志中找到故障	grep、管道、首尾查看、计数	15
Q5	修复启动配置	精确替换、备份、输出通道	15
Q6	安装交接工具	本地 Debian 包、安装数据库	10
Q7	远程连接与日志回收	SSH、公钥认证、SCP	10
Q8	昨天还能运行	进程、信号、端口、HTTP	20
Bonus	保留调试会话	tmux、后台运行与清理	+10

建议按 Q1→Q8 的顺序完成，但各题独立评分，不依赖其他题的产物。

0.1 统一接口

每题分别对应 `src/main/q{1-8}.sh`，接受两个参数：

```
bash src/main/qN.sh INPUT_DIRECTORY OUTPUT_DIRECTORY
```

两者都是绝对路径，可能包含空格。输入目录已经准备好，输出目录已经存在；Q3 的输出中还预置了待删除缓存。评测器从任意当前工作目录调用脚本。使用骨架里的 `input`、`output`，不要写死路径。只有 Q8 与 Bonus 额外接收动作参数。

输入素材不可修改，只在本题输出目录创建和修改产物。Q8 / Bonus 还允许管理题面指定的训练进程和会话。各个脚本必须保留、使用 Bash 编写，不调用 Python、其他编程语言或预存答案替代命令；可以调用题目指定的教师工具。

非空文本使用 UTF-8、LF 换行，末尾保留一个换行；空结果为 0 字节。不在文本中添加进度提示、行号或其他说明。Q1–Q6 将连续执行两次，第二次应保持产物内容和权限稳定；Q6 的 dpkg 内部锁、备份数据库等维护状态不做逐字比较，安装状态与实际工具会重新检查。

未实现的骨架退出状态为 77，记 0 分。实现时删除提示和 `exit 77`；正常完成退出 0，意外错误返回非零。单次脚本调用限制为 15 秒，因此不能留下等待输入的命令、无限 `tail -f` 或持续前台服务。

0.2 环境变量文件

Q5、Q7、Q8 和 Bonus 会在输入目录放置 `.env` 文件。它们由教师生成，使用可被 Bash 读取的引用格式。可以这样读取：

```
source "$input/remote.env"
```

这只是读取本题参数，不是读取个人配置。临时用户名、路径、PID、端口和会话标识均以这些文件为准。

0.3 我们如何打分

正如前面所说，我们会使用 GitHub CI Runner 来进行评分。你提交的 bash 文件们会出现在一台我们创建并固化了环境的 Ubuntu 24.04 上。我们会依次执行所有的 bash 文件，并观察是否达成了我们期待的效果。

每个题目的分值分布如上。本次考核我们并不要求 AI 使用记录提交，但是我们真的建议把这些命令背出来。

Q1 找到开发目录 · 10 分

> **Q1 · 就在那个目录里**
> “配置在哪?”
> “ws 里面。”
> “ws 是啥”
> 对方看了一眼你的终端，露出了一种 “这也需要解释吗” 的表情。问题是，你刚才为了找文件已经来回切了六次目录，现在连自己在哪都不知道了。这个时候再问一次，双方都会有一点尴尬。
> 更尴尬的是，对方坚称文件就在那，你却怎么都看不到。你已经准备在群里宣布交接包不完整，他走过来敲了一个带参数的命令，那个以点开头的文件就出现了。
> 文件没丢。你们只是对 “看一眼目录” 这件事有着不同的理解。
> 作为今晚的接班人，先给自己留一份准确的位置和编号记录。否则半小时以后你会再次回来问，而他会再次说：“就在那个目录里。”
> BTW，ws 是 Workspace 的缩写，这个概念我们会在讲 ROS2 的时候讲到。

答题文件： `src/main/q1.sh`。

输入结构：

```
INPUT/
+-- handover/
|   +-- .robot-id
|   `-- README.txt
`-- notes/
    `-- welcome.txt
```

要求：

1. 进入输入目录的 `handover/`，把该目录的绝对路径保存为输出目录中的 `location.txt`。
2. 将隐藏文件 `.robot-id` 的完整内容保存为 `robot-id.txt`。
3. 将 `notes/welcome.txt` 的完整内容保存为 `welcome.txt`。

编号和欢迎文本会变化，根路径包含空格，调用时的当前目录不保证是仓库。`location.txt` 是本次真实绝对路径，而不是示例路径。

评分： 路径 4 分，隐藏编号 3 分，欢迎文本 3 分。

相关命令： `pwd`、`cd`、`ls`、`cat`、重定向。可以用 `man` 或 `--help` 查用法，不考 Linux 历史背诵。

Q2 建立哨兵工作区 · 10 分

> **Q2 · 最终版的最终版**
> 找到目录以后，你看到了一些令人安心的名字：final、final2、final-new，以及一个修改时间比它们都新的 old。队友解释说，old 才是现在用的版本，final 是上次比赛用的，final-new 是一个没做完的尝试。至于 final2，他也不太记得了，建议你“先别删，万一有用”。
> 我相信，起这些名字的时候，每一位作者都真心认为自己终于做完了。只不过下一次调试来得比他们的命名规范更快。
> 现在我们给你一个重新开始的机会：脚本放在脚本目录，配置放在配置目录，日志有自己的位置，临时文件也有一个明确的去处。
> 先让启动脚本说一句 Hello。暂时不要求它带着机器人去打比赛，它能从正确的位置被找到，就已经比刚才那四个最终版好一点了。

答题文件： `src/main/q2.sh`。输入目录没有内容依赖。

在输出目录建立：

```
OUTPUT/
`-- sentry_ws/
    +-- scripts/
    |   `-- hello.sh
    +-- config/
    |   `-- robot.conf
    +-- logs/
    |   `-- .keep
    `-- tmp/
        `-- scratch.txt
```

`.keep`、`scratch.txt` 均为空文件。`hello.sh` 完整内容为：

```
#!/bin/bash
echo "Hello, AIM Linux!"
```

`robot.conf` 完整内容为：

```
robot=sentry
mode=training
log_level=INFO
```

所有文件必须为普通文件，不能用符号链接代替。脚本此时不要求有执行权限。再次运行不能把文本重复追加。

评分：四个子目录 4 分，两个空文件 2 分，脚本内容 2 分，配置内容 2 分。

相关命令： `mkdir`、`touch`、`cat` 和多行重定向。区分 `>` 与 `>>`。

Q3 整理配置与权限 · 10 分

> **Q3 · 万能的 777**
> 目录终于整理好了。你运行启动脚本，终端回了一句 Permission denied。
> 旁边立刻有人给出建议："权限问题？全改成 777。"说这句话的时候，他甚至没有抬头，手上还在给另一台机器人调参数。
> 这个办法确实很热情。它把所有人的读、写、执行权限一起送了出去，仿佛给实验室大门贴了一张"欢迎随时改我配置"的告示。你问为什么需要这么多权限，他说："不知道，反正我一直这么用。"
> 这也是很多祖传环境的诞生方式：一个临时办法跑通了，于是临时了两年。
> 这真的给我们造成了很多麻烦。777 和 sudo 就像两个不死不休的魔咒，虽然 sudo 很方便，但是。。这能对吗??

答题文件： `src/main/q3.sh`。

输入目录提供 `hello.sh`、`robot.conf`、`draft.conf` 和含 `cache.txt` 的 `tmp/`。脚本内容与 Q2 相同，初始没有执行权限。两份配置内容随测试变化。首次调用时，输出目录已经有 `sentry_ws/tmp/cache.txt`，模拟上次运行留下的缓存；必须实际删除它。

要求：

1. 在输出目录整理 `sentry_ws/`，把脚本复制到 `scripts/`，两份配置复制到 `config/`；可以复制输入的临时素材练习整理，但最终不保留 tmp。
2. 保留正式配置 `config/robot.conf`，另在 `backup/robot.conf` 保存完整副本。
3. 把 `config/draft.conf` 移动并改名为 `archive/draft.conf.old`，原位置不再保留。
4. 删除输出工作区的整个 `tmp/`，保留其他文件。
5. 将 `scripts/hello.sh` 权限设为 `750`，`config/robot.conf` 权限设为 `640`。
6. **直接执行**工作区中的 `hello.sh`，将标准输出保存为输出根目录的 `hello-output.txt`，完整内容为一行 `Hello, AIM Linux!`。

源素材不能改变；第二次运行同样应成功。无需修改文件所有者或使用 sudo。

评分： 复制及备份 3 分，移动 2 分，删除 1 分，两个权限共 2 分，执行与输出 2 分。

相关命令： `cp`、`mv`、`rm`、`chmod`。数字权限与"所有者 / 组 / 其他人"三类权限对应；不是所有权限问题都需要 root。

Q4 从日志中找到故障 · 15 分

> **Q4 · 它很努力地告诉你它炸了**
> Python 题面已经抱怨过五十万行日志了。到了真正的开发机上，你会发现日志还有第二种令人崩溃的能力：它会非常认真地误导一个着急的人。
> 有人写 “ERROR: device unavailable”，有人写 “previous ERROR has been handled”，还有人每秒钟打印一次 Heartbeat。最后这种最积极，程序停止之前它一直都觉得自己挺好的。
> 你搜了一遍 ERROR，屏幕红了一大片。队友凑过来看，说：“这么多错误?”
> 当然，日志作者还会补充一句：“你看级别啊，我这是 INFO。”
> 他没有说错。只不过在凌晨两点，大家对阅读理解的耐心往往已经不太充足。
> 欢迎来到 ros2 logs 的海洋。

答题文件： `src/main/q4.sh`。

输入为 `boot.log`、`run.log`。非空行格式固定：

```
级别 时间 模块 消息
```

前三个字段之间各一个空格；级别只有大写 `INFO`、`WARN`、`ERROR`，消息可以包含空格。输入没有空白行，但整个文件可以为空。两份日志各自已按时间排序，boot 整体早于 run。

示例：

```
ERROR 09:00:02 camera device unavailable
INFO 09:00:03 launcher previous ERROR has been handled
```

第一行是错误，第二行不是。

输出要求：

文件	内容
`all.log`	按 boot、run 顺序完整拼接
`errors.log`	级别字段等于 ERROR 的完整行
`attention.log`	级别字段等于 WARN 或 ERROR 的完整行
`recent.log`	all 最后 5 行，不足则全部
`first-errors.log`	errors 最前 3 行，不足则全部
`error-count.txt`	错误行数，只有整数与换行

保持原行内容、顺序和重复行；不要排序或去重。没有匹配时提取文件为空，计数为 `0`，脚本仍正常结束。公开测试包含常规、无错误、全空和不足指定行数的日志。

评分： 合并 2 分，错误 4 分，告警 3 分，末尾 2 分，首条错误 2 分，计数 2 分。

相关命令： `cat`、`grep`、`head`、`tail`、`wc`、管道。普通正则和扩展正则的语法不同；`grep` 无匹配返回 1，不应当作本题致命错误。

PS：关于 grep 没有匹配结果

骨架中的 `set -e` 会在命令返回非零状态时提前结束脚本。但 `grep` 没有找到匹配行时返回 1，这是本题允许的正常情况。例如，提取错误日志时可以写成：

```
cat "$output/all.log" | grep "^ERROR" > "$output/errors.log" || true
```

`|| true` 表示左边的命令或管道未成功时，执行一个返回成功状态的命令，让脚本继续。没有错误行时，重定向仍会创建一个空的 `errors.log`，后面应继续生成其他结果，并将错误计数写为 `0`。

注意：`|| true` 也会忽略文件不存在、权限不足等真正的错误。使用前确认输入文件存在且可读，只在本题允许“没有匹配行”的搜索处使用，不要给整个脚本或所有命令都加上它。

Q5 修复启动配置 · 15 分

> **Q5 · 我改了，真的改了**
> 日志告诉你端口不对。队友马上打开配置文件，熟练地找到那行数字，改掉，保存，make restart。
> 没有变化。
> 他又改了一遍。还是没有变化。接着他开始怀疑程序没有重新读取配置、编辑器没有保存，甚至怀疑 Linux 正在偷偷和他作对。最后你指了指行首那个小小的井号。
> 他沉默了一会儿，说："这行怎么是注释?"
> 我们无法得知它是怎么变成注释的，但它确实已经是注释很久了。又因为配置文件一般在.gitignore 里，我们无法责怪当事人。
> 另一位队友提出全局替换。AI 也很乐意帮忙，顺便把备用端口和说明里的示例一起改成了同一个数字。配置文件看起来特别统一，下一次排查时你们连原来打算用什么都不知道了。
> 这个时候你可能回忆起了某次培训某位学长痛骂 magic number 的那个下午，但是好像已经来不及了。

答题文件： `src/main/q5.sh`。

输入有 `robot.conf` 和 `tools.env`。配置中的有效 `port`、`log_level` 各出现一次；键和值之间无空格，旧值会变化。注释、空行和相似键不是修改对象，例如：

```
# port=9000

port=9000
backup_port=9000
# log_level=DEBUG
log_level=DEBUG
```

要求：

1. 完整备份为 `robot.conf.bak`。
2. 生成输出 `robot.conf`，仅把有效 `port` 值改为 `8080`、有效 `log_level` 值改为 `INFO`。其他行、顺序、注释和空行保持不变。
3. 读取 `tools.env` 中的 `CHECK_TOOL`。调用接口为 `bash "$CHECK_TOOL" CONFIG_FILE`，检查修复后的文件。
4. 工具标准输出保存为 `check.out`，标准错误保存为 `check.err`，不要混合或追加上次内容。

工具成功时退出 0，stdout 为一行 `CONFIG OK`，stderr 为一行 `NOTICE: training check only`；该提示不表示失败。配置不合格时退出非零，工具不修改文件、不连接网络。

评分： 备份、端口、日志等级、其他内容保留、通道分流各 3 分。

相关命令： `cp`、`sed`、`>`、`2>`，以及退出状态。输入遵守上述格式，不考重复键、缺失键或任意配置语法解析。

Q6 安装交接工具 · 10 分

> **Q6 · 满格的 WiFi**
> 你缺一个调试工具。平时在实验室，这件事只需要一句安装命令，等一会儿，然后接着干活。
> 但是去了备场就不一定了。WiFi 的信号是满格的，网页偶尔也能打开，软件包下载却停在一个非常坚定的进度上。你看着它，它看着你，谁也没有继续往前走。
> 旁边的人说：“要不连我热点?”五分钟后，整个软件组都连上了他的热点。他开始认真计算自己的流量还够不够撑到比赛结束。
> 幸好，交接包里还有一个本地安装包。有人把它解压开，看到里面的可执行文件，立刻宣布：“装好了。”包管理器对此持不同意见。
> 看见文件、完成安装、确认能运行，这三件事需要分别弄清楚。今晚先用这个不依赖网络的小工具把流程走通，下次到赛场时，希望你还能想起自己把它装在了哪里。
> P.S. 这玩意是个没有什么意义的安装包，不是什么神奇的调试工具。。

答题文件： `src/main/q6.sh`。

输入提供 `aim-sentry-tools.deb`，包名为 `aim-sentry-tools`，架构为 all，版本随测试变化；没有依赖和安装维护脚本，包内有 `/usr/bin/aim-sentry-info`。

要求：

1. 使用 dpkg 把包安装到输出目录的 `sysroot/`，不能安装到真实系统根目录。
2. 安装数据库位于 `sysroot/var/lib/dpkg/`，必须记录正确版本的 `install ok installed` 状态。
3. 运行安装后的 `sysroot/usr/bin/aim-sentry-info`，stdout 保存为 `tool-output.txt`。工具输出 `SENTRY TOOL 版本号`，版本须来自实际包。

教师用普通账户执行本题。dpkg 支持 `--root= 目录` 指定安装根、`--force-not-root` 允许非 root 安装；日志可以用 `--log=/dev/null` 关闭。这些选项用于本题的无维护脚本训练包。不要使用 sudo，不联网安装或刷新源。只解压 `.deb`、手写数据库或直接写报告都不能代替安装。

允许再次安装相同版本；dpkg 的内部数据库备份不是作业产物规格。测试器独立查询数据库并运行真实安装文件。

评分： 正确安装状态与版本 6 分，真实工具运行及记录 4 分。

相关命令： `dpkg`、`dpkg-query`。课程中的 apt 在有软件源时负责依赖解析和下载；本题刻意使用本地无依赖包，以免外部镜像站决定成绩。

Q7 远程连接与日志回收 · 10 分

> **Q7 · 你连的是谁**
> “把机器人上的日志发我一下。”
> “发了。”
> 对方打开文件，里面是你十分钟前在自己电脑上跑出来的输出。你说程序应该一样，他问你那为什么机器人还是不动。
> 这台哨兵没有屏幕，调试时大家围着它蹲一圈，最后还是要回到各自的笔记本前。SSH 很方便，也很容易让一排长得差不多的终端把人绕进去：这个是本机，那个是开发机，刚才断开的才是机器人。
> 有一次你改了半天配置，重新启动也毫无变化。仔细看了眼主机名，才发现刚才一直在修自己的虚拟机。机器人在旁边，甚至不知道有人正在努力拯救它。
> 这次先确认连接对象，用公钥完成登录，再把远端的文件真正取回来。日志作者在哪台机器上受过苦，读日志的人至少应该弄清楚。

答题文件： `src/main/q7.sh`。

教师启动只监听回环地址的临时 SSH 服务，使用独立远端账户。输入目录 `remote.env` 提供：

变量	意义
`HOST`、`PORT`	目标地址与动态端口
`REMOTE_USER`	远端临时用户名
`KEY_PATH`	教师提供的客户端私钥路径
`KNOWN_HOSTS`	已核对的目标主机公钥记录
`REMOTE_LOG`	远端日志的绝对路径

要求：

1. 在输出目录创建权限为 `600` 的 `ssh_config`，定义别名 `aim-sentry`；地址、端口、用户、密钥、已知主机文件均读取给定参数。
2. 配置 `IdentitiesOnly yes` 和 `StrictHostKeyChecking yes`。不得关闭主机密钥检查，也不要依赖个人 SSH 配置或 agent；用 `BatchMode yes` 或命令行等价选项避免等待密码输入。
3. 使用该配置和别名通过 SSH 执行远端 `whoami`，将返回的用户名保存为 `remote-user.txt`。
4. 使用 SCP 将 `REMOTE_LOG` 下载为输出目录的 `received/errors.log`。

客户端脚本没有本地读取该远端私有日志的权限。远端服务只接受公钥认证；教师会用输出配置重新连接，并比较真实远端文件与下载内容，同时核对服务端连接与文件传输会话。

密钥、用户名、端口和日志内容都会变化。路径可能包含空格，SSH 配置中的路径也要正确引用。不要复制或提交临时私钥。

评分： 配置 3 分，远端命令与复连 3 分，SCP 文件传输 4 分。

相关命令： `ssh -F`、`scp -F`、公钥认证、known_hosts。OpenSSH 新版 SCP 默认使用 SFTP 子系统，这是正常行为。

Q8 昨天还能运行 · 20 分

> **Q8 · 昨天还能运行**
> 到这里，文件找到了，权限正常了，工具也装好了。你甚至已经成功连上了另一台机器。按照原计划，现在应该可以回去睡觉。
> 这时队友过来，指着一个打不开的状态页面，说出了那句熟悉的话：“It works on my computer.”
> 你查了一眼进程，发现日志采集器居然有两个。有人建议把所有 Python 进程一起结束，再重新启动。旁边刚开始录数据的队友缓缓抬起了头。这个建议获得了一次非常及时的否决。
> 另一个人 ping 了一下，宣布网络正常。第三个人准备重启整台机器。你问服务到底监听哪个端口，大家终于安静了一会儿。
> 这就是调试中很常见的场面：每个人都在提供办法，暂时还没有人提供证据。
> 今晚先找准进程，留几条日志，正常停止需要停止的那个，再把服务恢复到正确的地址和路径。等页面真的返回了预期内容，我们再决定谁去拿楼下的外卖。

答题文件： `src/main/q8.sh`。

本题调用两次：

```
bash src/main/q8.sh INPUT OUTPUT repair
bash src/main/q8.sh INPUT OUTPUT cleanup
```

省略动作时默认为 repair。repair 返回后，服务必须保持运行，供教师独立请求；cleanup 返回后服务必须停止。

教师预先启动目标日志采集器、另一个不能停止的采集器、一个错误 HTTP 服务。输入有 `server.conf` 和 `process.env`：

变量	意义
`TARGET_PID`、`TARGET_TOKEN`	目标采集器 PID 和实例标识
`WORKER_LOG`	目标采集器不断增长的日志
`SERVICE_PID`	原错误 HTTP 服务 PID
`TARGET_PORT`	修复后应使用的动态端口
`START_TOOL`	教师服务启动工具路径

A. 定位与停止采集器 · 10 分

repair 阶段要求：

1. 用进程查询命令查看目标 PID，保存输出根目录的 `process.txt`，每行一个键：

```
pid= 实际 PID
user= 实际所有者
state= 实际进程状态
command= 完整命令行
```

state 接受实际 ps 状态字符串，如 `S` 或 `Ss`；进程应处于 R / S 状态。命令行应包含教师给定实例标识，不能写死用户名和 PID。

2. 保存至少两条目标采集器的 TICK 日志为 `collector-sample.log`，允许包含 START 行。
3. 向目标 PID 发送普通终止信号，等待它正常退出。教师程序正常退出时会写出 `STOP 实例标识`。
4. 另一个采集器必须继续运行，不能用 killall 或宽泛的 pkill 代替 PID 定位。

评分：识别记录 3 分，正常停止 3 分，保留另一个进程 2 分，日志记录 2 分。教师检查真实进程和停止记录，不仅检查学生写的文本。

B. 恢复 HTTP 服务 · 10 分

两个公开场景分别是端口错误和路由错误，教师会逐个验证。输入 `server.conf` 固定包含三行：

```
bind=127.0.0.1
port= 原端口
route= 原路径
```

repair 阶段要求：

1. 正常停止 `SERVICE_PID` 指定的原服务，等待退出。
2. 生成输出 `server.conf`，完整内容是 `bind=127.0.0.1`、`port=TARGET_PORT 实际数值`、`route=/status` 三行，保持这一顺序。
3. 使用教师启动工具运行修复后的服务：

```
bash "$START_TOOL" "$output/server.conf" "$output/service"
```

工具启动后台进程后返回，写入 `service/pid` 和 `service/service.log`。工具只启动，不替你等待服务就绪或验证 HTTP。

4. 等待就绪，用 curl 请求 `http://127.0.0.1:TARGET_PORT/status`。保存 `http-status.txt` 为一行 `200`，`http-body.txt` 为一行 `SENTRY READY`。使用有界重试和超时，不能无限等待。
5. 返回时保留服务运行。教师独立发出请求，不信任学生报告中的 200。

cleanup 阶段要求：根据本题服务状态目录定位修复后的服务，正常停止它并等待端口释放；保留已生成的报告，不操作其他进程。

评分：原服务停止与新配置 3 分，真实 HTTP 响应和报告 4 分，cleanup 释放端口 3 分。

相关命令： `ps`、`kill`、`cat` / `tail`、`ss`、`curl`。ping 成功只说明部分网络可达，不证明 HTTP 程序正确。不要修改系统路由、防火墙或真实 SSH 服务。

Bonus 保留调试会话 · +10 分

> **Bonus · 人可以下班，日志还得接着写**
> 状态页面终于恢复了。你准备让采集器再跑一会儿，观察问题会不会重新出现。
> 然后笔记本合上了。又或者校园网络断了一下。总之，你回来的时候，SSH 已经断开，刚才那个终端也不在了。你重新连进去，发现自己还要先确认采集器究竟活没活着。
> 这一天，你已经为太多没有必要重来的事情重来过一次。
> 所以我们给调试留一个能继续工作的会话：终端可以退出，连接可以重建，程序还留在原来的地方。第二天回来，你能接着看同一个进程写下的日志，而不是再说一次“我们重新跑一遍吧”。
> 当然，真正结束调试时还是要收拾。tmux 帮你保留工作，不负责替你判断实验室里哪些进程已经没人记得了。

答题文件： `src/main/bonus.sh`，额外接收 start / stop 动作，默认 start。

输入 `bonus.env` 提供 `TMUX_SOCKET`、`SESSION`、`TOKEN`、`WORKER_TOOL`。所有 tmux 命令使用 `-L "$TMUX_SOCKET"`，只管理指定会话。

start 阶段：

1. 创建指定名称的分离会话，运行教师采集器，接口为 `python3 "$WORKER_TOOL" --token "$TOKEN" --log "$output/bonus.log"`。
2. 脚本返回后会话无附着客户端，采集器继续运行，日志继续增长；全程只启动一个实例。

stop 阶段：正常停止会话内采集器，再结束指定会话；不结束其他会话。

评分： 分离会话 4 分，返回后日志继续增长 4 分，停止和清理 2 分。本题用真实 tmux 会话验证持久运行，不要求模拟公网断线。

9. 评分与提交

9.1 唯一评分通道

全部权重、生成数据和检查代码公开在 `tests/coursework.py`。每个小项需要通过该题所有公开场景才得到该小项分数，部分小项失败不取消其他小项分数。

必做总分 100，Bonus 单独最多 10。通过条件是**必做至少 60 分，且 Q7、Q8 的全部检查通过**。Bonus 不抵消这两项关键能力。满分条件是必做全部小项通过，不要求 Bonus。

没有隐藏测试、单独 grader、人工评阅或现场答辩。CI 只验证提交脚本的行为，不能证明作者独立理解全部命令；允许 AI 辅助，不设置无法自动判断的主观分。

原始模板的脚本全部未实现，初始化可成功，但 `assessment_passed` 为 false、得分为 0。开始答题后，未达到通过条件，完整 CI 退出失败；Summary 仍展示分数。单题本地测试不产生正式通过结论。

9.2 合法修改与提交

只允许修改 `src/main/q1.sh`–`q8.sh`、`bonus.sh`，以及可选 `.agent-sessions/` 归档。其余文件不可修改；所有答题文件必须保留为普通文件。不要提交 generated、个人 SSH 密钥或密码。

使用模板创建个人仓库，在 main 开发；每完成一段任务可以提交并推送。CI 会直接在 Ubuntu runner 上运行，无需 Docker。累计修改以初始模板提交为基准检查，测试从该基准取出；不要重建 Git 历史。

正式提交内容：个人仓库地址、指定 main commit 链接、与该 commit 对应的完整 Actions run 链接。不要只提交单题自测结果或旧 commit 的成功截图。提交入口和截止日期由培训负责人公布。

9.3 学习参考

- 2526 Linux 作业：`https://github.com/unnc-aim/aim-2526-linux-coursework`
- 2627 Linux 课程：`https://github.com/unnc-aim/aim-2627-rookie-courses/tree/main/Computing/02-Linux`
- Linux2：路径与目录；Linux3 / 4：文件与权限；Linux5：包管理。
- Linux6：SSH；Linux7：文本处理；Linux8：进程与 tmux；Linux9：网络与服务。
- 不熟悉的命令先查 `man 命令` 或 `命令 --help`。先用小样例理解输出，再组合命令。

> **尾声 · 给下一位队友留一句有用的话**
> 天快亮的时候，那个 foxglove 的 message 终于正常的开始吐了。
> 它没有很响，也没有让哨兵突然冲出去巡逻。屏幕上只是多了一行很普通的字，日志还在继续写，几个应该退出的进程也确实退出了。对已经围着终端坐了一晚的人来说，这样就挺好。
> 你开始写交接说明。这次里面有目录、有配置、有日志的位置、有连接方式，也有几个需要留意的进程。你想了想，删掉了开头那句 “It just works.”。
> 它可能是真的，但它帮不了下一位队友。
> 等到下一次备场很吵、网络很差、身后有人不断问还有多久的时候，你大概还是会紧张。至少这一次，你知道自己在什么位置、连着哪台机器、改了哪个文件，也知道该去哪里找它留下的证据。
> 2027，希望哨兵动起来。
> 如果它还是没有动，也希望你能够给接手的人留下一句比 “我这边没问题” 更有用的话。